

Packet-Detection-Lab Write-Up

Tahmid Abtahee · Independent Study, COP 4900, University of South Florida · Supervised by Dr. Amanda Holloman · September 25, 2026

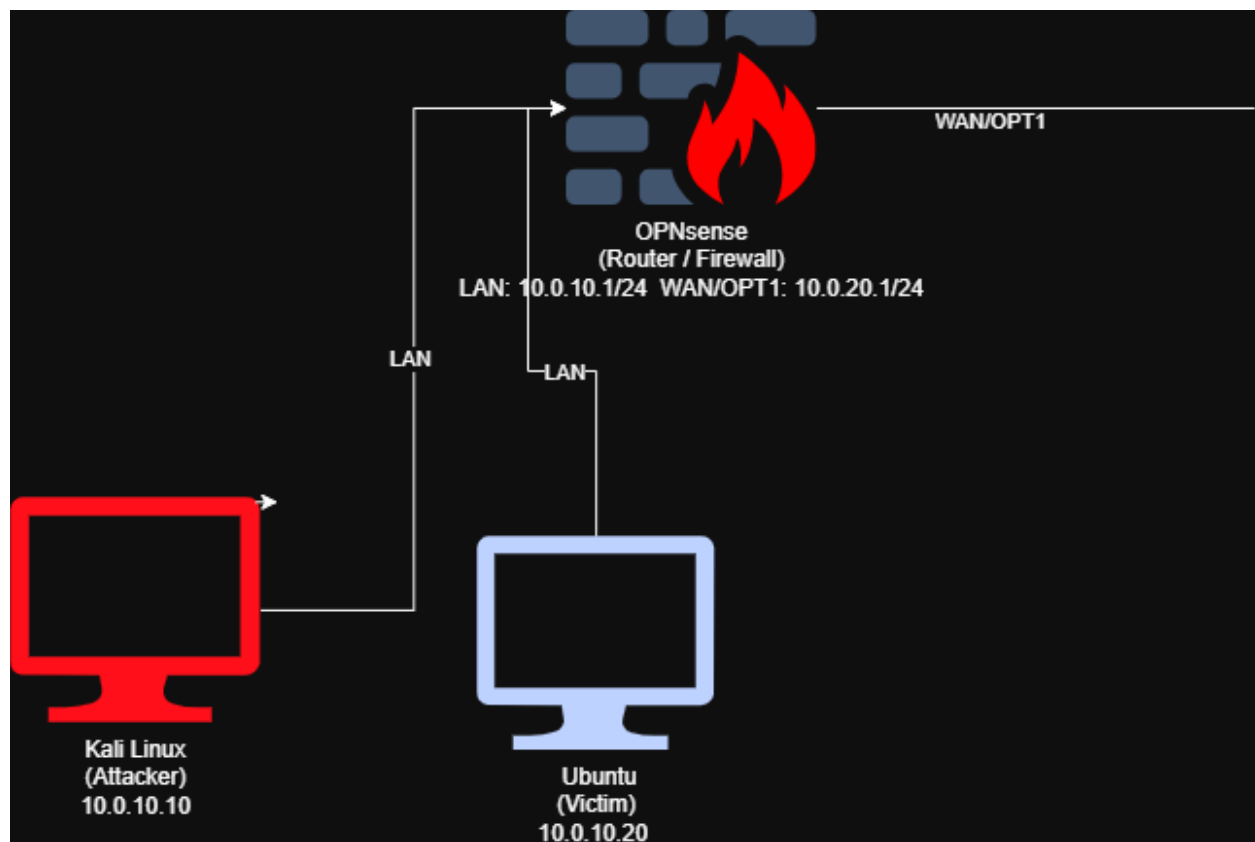
Why I Built This

Prior to this project, my detection portfolio consisted entirely of host-based and cloud API log telemetry: EDR-style alerts and cloud audit logs, that layer of visibility. That gap became apparent during a CISO interview, where it became clear I had no packet-level story to tell. This lab was built to close that gap: to demonstrate the ability to read and reason about raw network traffic at the byte level, rather than simply running a tool and interpreting its output.

The scope was deliberately narrow: one clean example per technique rather than repeated variations. The objective was functional understanding and familiarity with these networking concepts, as well as a stronger grasp of OSI layer concepts.

Lab Environment

OPNsense sits in the middle of the lab as a router and firewall, splitting two segments. Segment-a (LAN, 10.0.10.0/24) holds both the Kali attacker box (10.0.10.10) and the Ubuntu victim box (10.0.10.20) side by side. Segment-b (WAN/OPT1, 10.0.20.0/24) is kept separate for isolation. A temporary NAT-connected adapter was attached to the VMs only when installing packages, then removed to preserve isolation between runs.



Lab topology: OPNsense routes and separates segment-a (Kali attacker, Ubuntu victim) from the isolated segment-b (WAN/OPT1)

Tooling: Wireshark for packet capture and manual byte-level analysis, Suricata for signature-based detection, OPNsense for routing and segmentation, Kali Linux as the attack platform, and mitmproxy for the TLS interception demonstration. Zeek was used once, early on, for a general baseline conn.log check to establish what normal traffic looked like before any attack traffic ran. It was not wired up per-attack and is not a load-bearing part of the detection story presented here.

A note on Suricata's architecture, since it shapes how every detection result below should be read: the original plan was to run Suricata as an OPNsense plugin, inline, in IPS mode. That plugin had a persistent bug in which no rules were loaded, regardless of how the rule files, permissions, or app-layer configuration were adjusted, and the issue was never resolved. I pivoted to a standalone Suricata install (v8.0.3, apt) on the Ubuntu victim VM instead. This means every result in this write-up is IDS, alert-only, not inline blocking. I present that pivot as a deliberate engineering decision rather than a hidden shortfall: when a tool does not work as documented, working around it and documenting the tradeoff is itself part of the job.

One additional environment detail worth recording: a stale `/var/run/suricata.pid` file allowed an old Suricata process to keep serving stale rules after a kill command, so restarts appeared successful when they had not actually taken effect. The fix became a strict sequence applied every time: confirm the running PID with `ps aux`, kill `-9` it, remove the pidfile, sleep two seconds, start with the `-D` flag, sleep two seconds again, then reconfirm the new PID had actually changed. This was a minor issue, but it cost meaningful debugging time before it was identified.

Attack 1: ARP Spoofing

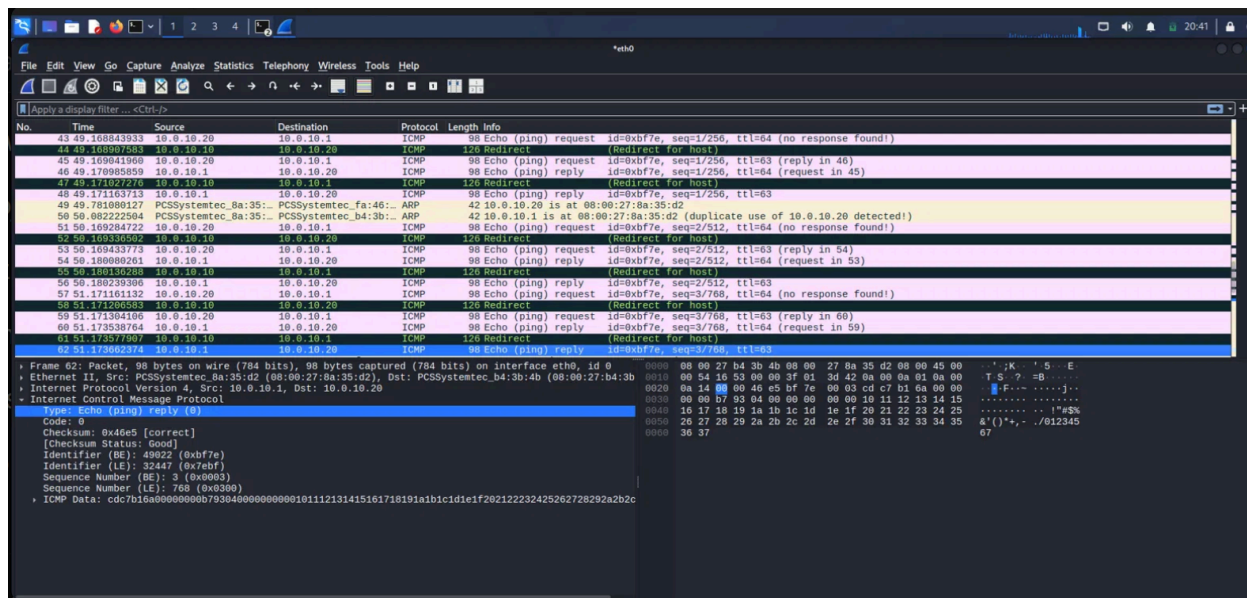
MITRE ATT&CK: T1557.002, Adversary-in-the-Middle: ARP Cache Poisoning

What I Did

Using arpspoof from the dsniff suite on Kali, I ran bidirectional poisoning between the OPNsense LAN gateway (10.0.10.1) and the Ubuntu victim (10.0.10.20), positioning the Kali box on-path between them.

What It Looked Like on the Wire

The capture (arpspoofing.pcapng, redone after an earlier run's capture was lost) shows gratuitous ARP replies going out roughly every two seconds, claiming the attacker's MAC owns both the gateway's IP and the victim's IP. That repetition is the tell: a real host resolves an address once and caches it. An attacker spoofing that address has to keep re-asserting the false mapping so it does not get overwritten or time out, so the traffic pattern itself, not just the content of any one packet, gives it away.



Wireshark's duplicate-IP-address expert info firing during the ARP spoofing capture

Detection

This is the one technique in the lab that Suricata genuinely cannot see. I confirmed this directly with `suricata --list-app-layer-protos`: ARP is not a supported protocol in Suricata 8.0.3 at all. Rules using alert arp fail to load with a "protocol not usable in a signature" error, regardless of how the ARP-detection YAML settings are configured. This is not a misconfiguration on my part; it is a real, documented limitation of signature-based IDS engines operating above layer 2. Detection instead came entirely from Wireshark's built-in expert-info warning, "Duplicate IP address configured!", which fires automatically once two different MAC addresses answer for the same IP within a capture.

Why This Matters

This is a legitimate finding, not a gap in my testing: ARP spoofing must be caught at layer 2, through Dynamic ARP Inspection, static ARP entries, or port security, because no amount of tuning enables a layer-3/4 signature engine to see traffic that never reaches IP.

Attack 2: DNS Tunneling

MITRE ATT&CK: T1071.004 (Application Layer Protocol: DNS), T1572 (Protocol Tunneling)

What I Did

iodine/iodined tunneling a fake zone, tunnel.local. Kali ran the iodined server side. The Ubuntu client connected with iodine -r -f 10.0.10.10 tunnel.local. The -r flag matters: it forces the tunnel into DNS-query mode specifically. Without it, iodine falls back to raw UDP once the handshake completes, which would bypass DNS-format detection entirely and defeat the point of the exercise.

What It Looked Like on the Wire

Captured in dnscapture.pcapng: 25 DNS queries carrying high-entropy, encoded subdomains of tunnel.local between the two hosts, clearly distinct from a normal, short, human-readable query pattern.

Detection

Working Suricata rule, sid 1000001, rev 5, running on the standalone Ubuntu Suricata instance:

```
alert udp any any -> any any (msg:"Possible DNS Tunneling";
  content:"|74 75 6e 6e 65 6c|"; sid:1000001; rev:5;)
```

That hex string decodes to the literal word "tunnel." This is a direct content match, not a length or entropy heuristic. It is a simple rule, stated plainly rather than overstated: it works because the fake zone name happens to appear in the query, and a real adversary using a different tunnel domain would slip past it untouched. A production-grade version of this rule would need to generalize to query length, label entropy, or NXDOMAIN rate rather than a fixed string. The rule fired repeatedly, confirmed in real eve.json alert entries.

```
teezy@ubuntu:~$ sudo grep '"event_type":"alert"' /var/log/suricata/eve.json | tail -3
```

```
{ "timestamp": "2026-09-24T06:37:49.567048+0000", "flow_id": "1398276485256319", "in_iface": "enp0s3", "event_type": "alert", "src_ip": "10.0.10.10", "dest_ip": "10.0.20.10", "dest_port": 47382, "proto": "UDP", "ip_v": 4, "pkt_src": "wired/pcap", "alert": { "action": "allowed", "gid": 1, "signature_id": 1000001 }, "category": "Sible DNS Tunneling", "severity": 33, "app_proto": "dns", "direction": "to_client", "flow": { "pkts_toserver": 40, "pkts_totclient": 44, "bytes_toclient": 4765, "start": "2026-09-24T06:35:48.849849+0000", "src_ip": "10.0.20.10", "dest_ip": "10.0.10.10", "src_port": 47382, "dest_port": 47382 }, "timestamp": "2026-09-24T06:37:53.571530+0000", "flow_id": "1398276485256319", "in_iface": "enp0s3", "event_type": "alert", "src_ip": "10.0.20.10", "dest_ip": "10.0.10.10", "dest_port": 53, "proto": "UDP", "ip_v": 4, "pkt_src": "wired/pcap", "alert": { "action": "allowed", "gid": 1, "signature_id": 1000001 }, "category": "Sible DNS Tunneling", "severity": 33, "app_proto": "dns", "direction": "to_server", "flow": { "pkts_toserver": 41, "pkts_totclient": 44, "bytes_toclient": 4765, "start": "2026-09-24T06:35:48.849849+0000", "src_ip": "10.0.20.10", "dest_ip": "10.0.10.10", "src_port": 47382, "dest_port": 53 }, "timestamp": "2026-09-24T06:37:53.580272+0000", "flow_id": "1398276485256319", "in_iface": "enp0s3", "event_type": "alert", "src_ip": "10.0.10.10", "dest_ip": "10.0.20.10", "dest_port": 47382, "proto": "UDP", "ip_v": 4, "pkt_src": "wired/pcap", "alert": { "action": "allowed", "gid": 1, "signature_id": 1000001 }, "category": "Sible DNS Tunneling", "severity": 33, "app_proto": "dns", "direction": "to_client", "flow": { "pkts_toserver": 41, "pkts_totclient": 44, "bytes_toclient": 4860, "start": "2026-09-24T06:35:48.849849+0000", "src_ip": "10.0.20.10", "dest_ip": "10.0.10.10", "src_port": 47382, "dest_port": 53 } }
```

Suricata's DNS tunneling rule firing repeatedly in eve.json

Attack 3: C2 Beacons

MITRE ATT&CK: T1071 (Application Layer Protocol), T1571 (Non-Standard Port)

What I Did

Plain Netcat standing in for a C2 channel, without emulating a more sophisticated framework. Kali ran a listener loop while true; do nc -lvp 4444; done, playing the role of a C2 server. Ubuntu beacons out on a timer, for i in 1 2 3 4 5; do nc -zv 10.0.10.10 4444; sleep 10; done, playing the role of a compromised host checking in. Port 4444 here is an arbitrary stand-in port, not meaningful on its own.

What It Looked Like on the Wire

Captured in c2-beacon-capture.pcapng: repeated short TCP connection attempts to the same destination and port, spaced by the sleep interval, with a plaintext "beacon: Ubuntu checking in" message riding along on some of the connections. The regularity of the interval, not any single packet's content, is what stands out against normal traffic.

Detection

Working Suricata rule, sid 1000005, rev 1:

```
alert tcp any any -> any 4444 (msg:"Possible C2 Beacons - Repeated
Connections"; flow:to_server; threshold:type both, track by_src,
count 3, seconds 60; sid:1000005; rev:1;)
```

The threshold clause is doing the interesting work here: it fires once after three connections land within sixty seconds, then suppresses repeats for the rest of that window rather than alerting on every one of the five connection attempts. This is a deliberate alert-fatigue reduction, the kind of tuning choice a SOC would actually want in production, not a limitation of the rule.

TLS Interception Demonstration

MITRE ATT&CK: T1557 (Adversary-in-the-Middle), T1040 (Network Sniffing, post-decryption)

What I Did

This step follows naturally from the ARP spoofing scenario: getting on-path is only useful if it lets you actually read something. mitmproxy ran on Kali (10.0.10.10:8080) as the interception point, targeting OPNsense's own HTTPS management GUI at https://10.0.10.1, since Kali had no route to the public internet in this isolated lab, and reconfiguring the VirtualBox networking to provide one was not justified by the tradeoff. Traffic was generated from the headless Ubuntu box with curl -k -x http://10.0.10.10:8080 https://10.0.10.1/, not a browser, since Ubuntu had no GUI. Because OPNsense's certificate is self-signed (RSA 4096-bit, negotiated over TLS 1.3 with TLS_AES_256_GCM_SHA384), mitmproxy needed to run with ssl_insecure=true to get past certificate validation.

What It Showed

mitmproxy's decrypted flow view revealed the OPNsense login page's full HTML, along with a PHPSESSID session cookie and an X-CSRFToken value, both in plaintext.

Session Actions Edit View Help

Flow Details
2026-09-25 18:26:56 GET https://10.0.10.1/ HTTP/2.0
+ 200 text/html 2.7k 202ms
Request Response Detail
set-cookie: PHPSESSID=0c83c79052e04c301273d7f6b4bdfef93; path=/; secure; HttpOnly; SameSite=Lax
set-cookie: PHPSESSID=0c83c79052e04c301273d7f6b4bdfef93; path=/; secure; HttpOnly
set-cookie: cookie_test=225aff74e0fab28c5c3c44085b3db219; expires=Fri, 25 Sep 2026 23:27:02 GMT; Max-Age=3600; path=/; secure; HttpOnly
expires: Thu, 19 Nov 1981 08:52:00 GMT
cache-control: no-store, no-cache, must-revalidate
pragma: no-cache
content-security-policy: default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline' 'unsafe-eval';
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
x-xss-protection: 1; mode=block
referrer-policy: same-origin
content-type: text/html; charset=UTF-8
accept-ranges: bytes
content-length: 2786
date: Fri, 25 Sep 2026 22:27:02 GMT
server: OPNsense
XML/HTML
[auto]
<!doctype html>
<html lang="en-US" class="no-js">
<head>
<meta charset="UTF-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="robots" content="noindex, nofollow" />
<meta name="keywords" content="" />
<meta name="description" content="" />
<meta name="copyright" content="" />
<meta name="viewport" content="width=device-width, initial-scale=1, minimum-scale=1" />
<meta name="mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-capable" content="yes">
<title>Login | OPNsense</title>
<link href="/ui/themes/opnsense/build/css/main.css?v=26434ed776d281bd" rel="stylesheet">
<link href="/ui/themes/opnsense/build/images/favicon.png?v=26434ed776d281bd" rel="shortcut icon">
<script src="/ui/js/jquery-3.5.1.min.js"></script>
<script src="/ui/js/theme.js?v=26434ed776d281bd"></script>
</script>
\$(document).ready(function() {
\$.ajaxSetup({
beforeSend: function(xhr) {
xhr.setRequestHeader("X-CSRFToken", "vbt6pwQQMYuoJ0xagi0oug");
}
});
}
[8080]
Flow: e Edit D Duplicate r Replay x Export d Delete b Save body _ Next flow p Prev flow
Proxy: ? Help q Back E Events O Options i Intercept f Filter w Save flows z Clear list

mitmproxy's decrypted response showing the OPNsense login page's session cookie and CSRF token in plaintext

Why This Matters

TLS 1.3 is a strong protocol, but it protects data between two endpoints that already trust each other. It does nothing against a machine physically sitting in the middle if that machine's certificate is accepted, whether through a compromised CA, a forced proxy, or a user clicking past a warning. This is the whole point of pairing this demonstration with the ARP spoofing scenario: ARP spoofing gets an attacker on-path, and this step shows what that position is actually worth once they are there.

Detection Engineering Summary

Two of the three attacks were genuinely detectable with Suricata; one, ARP spoofing, was not, for reasons rooted in the protocol itself rather than a tuning gap. That split is worth stating plainly rather than glossing over, because knowing what a tool cannot see is as much a part of detection engineering as knowing what it can.

Signature ID	Name	Trigger Logic	Scenario
1000001, rev 5	Possible DNS Tunneling	Hex content match on the literal string "tunnel" in the DNS query	Attack 2
1000005, rev 1	Possible C2 Beacons - Repeated Connections	Three or more TCP connections to the same destination port within 60 seconds, tracked by source, threshold-suppressed after firing once	Attack 3
n/a	Wireshark expert-info: Duplicate IP address configured	Built-in heuristic; two different MAC addresses answering for one IP. Only a viable detection path, since ARP is not a Suricata-supported protocol	Attack 1

MITRE ATT&CK Mapping

Scenario	Tactic	Technique	ID
ARP Spoofing	Credential Access / Collection	Adversary-in-the-Middle: ARP Cache Poisoning	T1557.002
DNS Tunneling	Command and Control	Application Layer Protocol: DNS	T1071.004
DNS Tunneling	Command and Control	Protocol Tunneling	T1572
C2 Beacons	Command and Control	Application Layer Protocol	T1071
C2 Beacons	Command and Control	Non-Standard Port	T1571
TLS Interception	Credential Access / Collection	Adversary-in-the-Middle	T1557
TLS Interception	Credential Access / Discovery	Network Sniffing	T1040

Lessons Learned

1. Layer matters more than tool sophistication. ARP spoofing was invisible to Suricata not because the rule was poorly written, but because ARP is not a protocol Suricata's engine handles at all. Knowing which layer a technique lives on tells you which tool can actually see it before time is spent tuning a rule that was never going to work.
2. When a tool breaks, document the pivot; do not hide it. The OPNsense Suricata plugin, which is never loading rules, despite genuine troubleshooting effort, could read as a failure. Pivoting to a standalone install and being upfront about losing inline blocking in exchange for a working, debuggable pipeline is a more honest and more useful story than presenting the original plan as if it had worked.
3. Simple detection rules have honest limits. The DNS tunneling rule matches a literal string. It works here because the fake domain name is known in advance, and it would miss a real adversary using a different one entirely. Writing the rule this way for a lab is reasonable; understanding exactly why it would not hold up in the field is the actual lesson.
4. Threshold logic is a deliberate design choice, not a compromise. The C2 rule firing once per three-in-sixty seconds rather than on every connection is exactly the kind of tuning a real SOC wants, since it reduces alert fatigue without losing the detection.
5. An on-path position is a force multiplier. ARP spoofing alone is inconvenient but not catastrophic, since traffic still flows. Paired with TLS interception, that same position turns into readable session cookies and CSRF tokens. The two attacks are more meaningful when demonstrated together than either is alone.